# CnA FRAMEWORK

Technical Architecture & Implementation Blueprint

| Specification | Baseline |
| --- | --- |
| Framework | CnA Framework |
| Runtime | PHP 8.4+; publish a maintained PHP support policy |
| Databases | MySQL, PostgreSQL and SQLite |
| Architecture | Strict modular packages; dependency inversion; service and repository layers |
| Core scope | HTTP kernel, DI container, CLI, database engine, query builder, ORM, security, AI, testing |
| Document purpose | Target architecture, implementation roadmap and production-readiness criteria |

Version 1.0 — Design baseline

This is an engineering specification for the intended framework. It does not claim that the framework has already been implemented, audited or security-certified.

# Document map


- 1. Vision and design principles
- 2. Layered architecture and request lifecycle
- 3. Package and repository structure
- 4. Core contracts and dependency injection
- 5. HTTP kernel, routing and middleware
- 6. Service and repository architecture
- 7. Database engine, query builder and ORM
- 8. N+1 prevention and query observability
- 9. Security architecture
- 10. Cache, queue, events and scheduler
- 11. Mail, notifications and storage
- 12. Provider-independent AI subsystem
- 13. CLI and developer experience
- 14. Configuration and deployment
- 15. Testing, quality gates and CI
- 16. Performance and scalability
- 17. Roadmap and release gates
- 18. Production-readiness checklist


# 1. Vision and design principles

CnA is designed as a complete modular PHP framework for APIs, web applications and long-running workers. It should offer a coherent developer experience without forcing every application to depend on a monolithic runtime. Packages communicate through stable contracts, and infrastructure implementations remain replaceable.

## Principles

- **Modularity:** each subsystem has a clear boundary, public API and independently testable implementation.
- **Dependency inversion:** application code depends on interfaces; adapters implement infrastructure contracts.
- **Explicitness:** transactions, authorization, configuration and side effects should be visible and auditable.
- **Secure defaults:** production defaults should be safe; development conveniences must be clearly isolated.
- **Database portability:** support three engines while documenting vendor-specific behavior.
- **Performance:** measurable query behavior, bounded resource use, caching and profiling.
- **AI as an optional subsystem:** pluggable providers, permissions, cost controls and observability.
- **Compatibility:** semantic versioning, deprecation windows and upgrade notes.

## Non-goals for the first release

- Do not clone every feature of established frameworks in the first milestone.
- Do not auto-rewrite arbitrary SQL to hide N+1 queries; detect and report instead.
- Do not make AI calls implicit in ordinary model/controller operations.
- Use OWASP ASVS Level 2 as a verification target, not as a certification claim.
- Do not promise identical SQL semantics across different database engines.

# 2. Layered architecture and request lifecycle

The HTTP layer translates network input into an application request. Controllers remain thin and delegate use cases to application services. Services coordinate business rules and transactions. Repository interfaces isolate persistence. Infrastructure adapters implement repositories using the database layer and ORM.

```
HTTP Request
↓
HTTP Kernel → Global Middleware → Router → Route Middleware
↓
Controller (transport adapter)
↓
Application Service / Use Case
↓
Domain Rules + Repository Interfaces
↓
Repository Implementations
↓
ORM / Query Builder / Database Connection
↓
MySQL | PostgreSQL | SQLite
```

Logging, cache, events, authorization, validation, mail and AI are injected through contracts rather than accessed through global static state.

| Layer | Responsibility |
|---|---|
| Kernel | Bootstraps providers, container, configuration and application lifecycle. |
| HTTP | Request/response objects, kernel, routing, middleware and exception mapping. |
| Application | Use cases, orchestration, DTOs and transaction boundaries. |
| Domain | Business rules and domain abstractions; no HTTP or database dependency. |
| Infrastructure | Database, cache, mail, queue, filesystem, AI and external-service adapters. |
| Console | CLI discovery, input/output, exit codes and command execution. |

# 3. Package and repository structure

Start with a Composer-compatible monorepo. It keeps coordinated development simple while allowing packages to be split and independently versioned later. Use the public namespace **CnA** and lowercase Composer names such as **cna/framework**.

```
cna-framework/
■■■ composer.json
■■■ cna # CLI executable
■■■ packages/
■ ■■■ contracts/ core/ container/ config/
■ ■■■ console/ http/ routing/ middleware/
■ ■■■ validation/ database/ query-builder/
■ ■■■ orm/ migrations/ auth/ authorization/
■ ■■■ security/ cache/ queue/ events/
■ ■■■ scheduler/ mail/ notifications/ storage/
■ ■■■ ai/ logging/ monitoring/ testing/
■■■ skeleton/
■ ■■■ app/Http/Controllers/
■ ■■■ app/Application/Services/
■ ■■■ app/Domain/
■ ■■■ app/Infrastructure/
■ ■■■ app/Models/
■ ■■■ bootstrap/ config/ database/migrations/
■ ■■■ public/index.php resources/ routes/
■ ■■■ storage/ tests/
■■■ docs/ tests/ benchmarks/
■■■ .github/workflows/
```

Every package should expose a small public API, keep implementation classes internal where possible, declare its own dependencies and include unit/contract tests. Avoid circular dependencies. The contracts package must never depend on concrete adapters.

# 4. Core contracts and dependency injection

The container should support constructor injection, interface bindings, singleton/scoped/transient lifetimes, tagged services, provider bootstrapping and circular-dependency detection. Avoid service-locator access in domain and application code.

```
namespace CnA\Contracts\Container;

interface ContainerInterface
{
public function bind(string $abstract, callable|string $concrete): void;
public function singleton(string $abstract, callable|string $concrete): void;
public function scoped(string $abstract, callable|string $concrete): void;
public function instance(string $abstract, object $instance): void;
public function get(string $abstract): object;
public function has(string $abstract): bool;
}
```

## Define contracts early

- ApplicationInterface, ServiceProviderInterface and BootableInterface
- LoggerInterface, ClockInterface and EventDispatcherInterface
- RequestInterface, ResponseInterface, MiddlewareInterface and RouterInterface
- ConnectionInterface, ConnectionManagerInterface, TransactionManagerInterface and QueryBuilderInterface
- CacheInterface, LockInterface and QueueInterface
- MailerInterface, StorageInterface and NotificationChannelInterface
- AIProviderInterface, EmbeddingProviderInterface and TokenCounterInterface

## Contract rules

- Use typed parameters/returns and immutable value objects where useful.
- Translate PDO/provider exceptions into documented framework exceptions.
- Expose optional capabilities through capability interfaces.
- Adopt PSR standards where practical: PSR-4, PSR-3, and PSR-7/15 interoperability.

# 5. HTTP kernel, routing and middleware

- Request/Response objects with normalized headers, query parameters, cookies, parsed bodies and uploaded files.
- Router supports verbs, named routes, groups, prefixes, constraints, route binding and URL generation.
- Middleware pipeline supports global, group and route-level middleware with deterministic ordering.
- Exception handler maps validation, authentication, authorization, not-found and unexpected exceptions to safe responses.
- JSON and content negotiation are first-class; HTML rendering may be an optional package.
- Trusted proxies must be configured explicitly; never trust forwarded headers from arbitrary clients.

## Example

```
$router->group(['prefix' => '/api/v1', 'middleware' => ['api']], function ($router) {
$router->get('/orders/{id}', [OrderController::class, 'show'])
->name('orders.show');
});
```

# 6. Service and repository architecture

Controllers handle transport concerns only: parse and validate input, invoke a use case, and map its result to a response. Business rules belong in domain objects and application services, not in controllers or ORM models.

```
final class CreateOrderService
{
public function __construct(
private OrderRepositoryInterface $orders,
private TransactionManagerInterface $transactions
```

```
) {}

public function execute(CreateOrderCommand $command): Order
{
return $this->transactions->run(function () use ($command) {
$order = Order::create($command);
$this->orders->save($order);
return $order;
});
}
}
```

- Repository interfaces belong near the application/domain boundary; SQL/ORM implementations belong in Infrastructure.
- Avoid generic repositories that expose every ORM method; prefer use-case-oriented methods.
- Repositories return entities or explicit read models, not raw database connections.
- The application use case owns transaction boundaries; repositories should not hide unrelated transactions.
- Use DTOs/commands for input and explicit serializers/resources for output.

# 7. Database engine, query builder and ORM

Build the database layer around PDO initially, with one connection adapter per driver and a dialect abstraction for SQL generation. The manager owns named connections, lifecycle, transaction nesting policy and optional read/write configuration. All values must be bound; identifiers must be quoted or validated by the active dialect.

| Component | Required capabilities |
|---|---|
| Connection manager | Named connections, lazy connection, reconnect policy and read/write split hooks. |
| PDO adapters | MySQL, PostgreSQL and SQLite; typed binding; exception translation. |
| SQL dialects | Identifier quoting, pagination, returning/insert IDs, JSON, upsert and schema differences. |
| Query builder | Select/insert/update/delete, joins, grouped where clauses, aggregates, ordering, pagination and subqueries. |
| Schema/migrations | Tables, columns, indexes, foreign keys, constraints and reversible migrations. |
| ORM | Models, casts, timestamps, relationships, eager loading, scopes and lifecycle hooks. |
| Transactions | Begin/commit/rollback, savepoints where supported and safe transient retry policy. |

## ORM relationship targets

- BelongsTo, HasOne, HasMany, BelongsToMany and polymorphic relationships.
- Nested and constrained eager loading.
- Attribute casts, guarded/fillable assignment controls, timestamps and optional soft deletes.
- Pagination, bulk operations, chunk/cursor iteration and bounded memory use.
- No hidden database access during serialization; relation loading must be visible and testable.

Maintain automated integration tests for all three engines. SQLite is useful for quick tests but is not proof of production parity. Document differences in JSON support, collations, isolation, locking and migrations.

# 8. N+1 prevention and query observability

Treat N+1 handling as a framework capability, not a marketing promise. Combine eager-loading APIs, development-time lazy-loading prevention, query listeners, repeated-query fingerprints, request-level budgets and telemetry.

- Attach request/job correlation IDs and collect duration, connection, redacted binding metadata, call site and normalized SQL fingerprint.
- Normalize SQL by replacing literal/bound values with placeholders; count repeated fingerprints per request/job.

- Flag repeated relation-shaped queries with changing foreign-key bindings and provide likely model/relation and source call site when available.
- Strict mode in development/tests should fail on unexpected lazy loads with actionable diagnostics.
- Support query budgets per route/job and slow-query thresholds; export structured logs and metrics.
- Redact secrets, personal data and sensitive bindings; never log credentials or raw sensitive payloads.

**Important:** Do not automatically rewrite arbitrary SQL into JOINs or eager loads. Query shape, cardinality and authorization semantics can change. Provide diagnostics and suggested fixes; let the developer choose the correct query.

# 9. Security architecture

Security is cross-cutting. Target OWASP ASVS Level 2 controls as an engineering verification baseline; this is not a certification or guarantee.

| Area | Baseline requirements |
|---|---|
| Authentication | Secure sessions, Argon2id password hashing where available, login throttling, reset tokens and MFA extension points. |
| Authorization | RBAC plus policy-based authorization; deny by default; enforce at use-case boundaries. |
| CSRF/XSS | CSRF tokens for cookie-authenticated state changes; context-aware output escaping and safe HTML handling. |
| SQL injection | Prepared statements, bound values, strict identifier allowlists and no unsafe interpolation. |
| Cryptography | Authenticated encryption (AEAD), secure randomness, key rotation/versioning and secret separation. |
| Sessions/tokens | Secure/HttpOnly/SameSite cookies; hashed tokens at rest; expiry, revocation and scopes. |
| Uploads | Size/type limits, content inspection, randomized names, non-executable storage and malware-scan hook. |
| Network safety | SSRF defenses, egress allowlists where possible, DNS/IP validation and redirect re-validation. |
| Headers | HSTS, CSP, frame restrictions, MIME sniffing protection, referrer and permissions policies. |
| Operations | Audit trail, dependency audit, secure configuration, rate limits, redacted errors and incident logs. |

- Never ship production secrets or hard-coded encryption keys.
- Disable debug mode in production and return generic errors while retaining protected diagnostics in logs.
- Use constant-time comparisons for secrets/tokens where applicable.
- Validate redirects, file paths, archive extraction paths and outbound network destinations.
- Protect password reset/recovery against enumeration and replay.
- Run dependency audits and static analysis in CI; maintain a vulnerability response process.

# 10. Cache, queue, events and scheduler

| Subsystem | Initial drivers and behavior |
|---|---|
| Cache | Array, filesystem, database; Redis adapter; TTL, locks, safe serialization and tags where supported. |
| Queue | Sync, database and Redis; retries, backoff, timeouts, failed-job store and idempotency guidance. |
| Events | In-process dispatcher first; queued listeners and transactional outbox integration as later capabilities. |
| Scheduler | Cron-triggered commands, overlap prevention, timezone handling, distributed locks and run history. |

Queue handlers should be idempotent where possible. Retry only safe failures; define maximum attempts and failed-job recovery. A database transaction and external queue publish are not automatically atomic—use an outbox pattern when correctness requires it.

# 11. Mail, notifications and file storage

- Mail abstraction with SMTP and optional SES, SendGrid or Mailgun adapters; templates, attachments, queue delivery and test fakes.
- Notification channels for mail, database and extensible external channels; user preferences and delivery receipts.
- Storage abstraction for local and S3-compatible object storage; visibility, signed URLs, streaming and lifecycle policies.
- External providers remain optional adapters; application services depend only on contracts.

# 12. Provider-independent AI subsystem

AI is an explicit application capability, not an implicit model feature. Stable contracts and provider adapters let applications change providers without rewriting business logic.

| Capability | Design target |
|---|---|
| Chat | Messages, roles, instructions, metadata and typed response objects. |
| Streaming | Provider-neutral events, cancellation and timeout handling. |
| Embeddings | Embedding interface, model metadata and vector-dimension validation. |
| RAG | Ingestion, chunking, metadata, retrieval, citations and access filtering. |
| Agents/tools | Explicit tool registry, schemas, permissions, execution limits and audit trail. |
| Memory | Application-controlled store with retention, deletion and tenant isolation. |
| Safety | Moderation hooks, prompt-injection defenses, data minimization and human approval. |
| Cost controls | Token accounting, budgets, model allowlists, tenant quotas and usage reports. |

## Provider and safety rules

- Provide vendor adapters behind framework-owned contracts, plus a fake provider for deterministic tests.
- Never give agents unrestricted shell, filesystem, database or network access by default.
- Each tool declares schema, permission, scope, timeout and audit behavior.
- Treat retrieved documents and tool output as untrusted data, not instructions.
- Enforce tenant access filtering before retrieval results reach a model.
- Require human confirmation for irreversible, financial, privileged or externally visible actions.
- Redact secrets and personal data before sending prompts to external providers where appropriate.

# 13. CLI and developer experience

The native executable is **php cna** or a project-local **./cna** wrapper. Commands should have consistent help, validation, exit codes, machine-readable output and dry-run support for destructive operations.

| Command family | Example commands |
|---|---|
| Project/generators | cna new; cna make:controller; cna make:service; cna make:repository; cna make:model |
| Database | cna db:status; cna migrate; cna migrate:rollback; cna migrate:refresh; cna db:seed |
| Cache | cna cache:clear; cna cache:warm |
| Queue | cna queue:work; cna queue:failed; cna queue:retry; cna queue:flush |
| Security | cna security:check; cna audit; cna key:generate |
| Diagnostics | cna route:list; cna config:check; cna db:profile; cna doctor |

| Command family | Example commands |
|---|---|
| AI | cna ai:providers; cna ai:diagnose; cna ai:usage |
| Operations | cna schedule:run; cna optimize; cna about |

Generators should follow the recommended architecture but avoid unnecessary boilerplate. Generate related request, service, repository, model and tests only when configured or requested.

# 14. Configuration and deployment

- Use typed configuration objects and validated environment-variable parsing at boot.
- Separate local, testing, staging and production configuration; never commit secrets.
- Provide health/readiness endpoints with minimal information disclosure.
- Document Nginx/Apache, PHP-FPM, OPcache and worker deployment.
- Use least-privilege filesystem ownership, secure log rotation and explicit writable paths.
- Support graceful shutdown for queue workers and scheduled tasks.
- Document backups, restore testing, migrations and rollback strategy.
- For horizontal scaling, externalize sessions, cache and queues; avoid local-disk assumptions for shared state.

# 15. Testing, quality gates and CI

| Test layer | What it proves |
|---|---|
| Unit | Small classes and domain rules behave correctly in isolation. |
| Integration | Database, cache, queue and provider adapters work against real services. |
| Contract | Every adapter obeys the same public interface semantics. |
| Architecture | No forbidden dependency direction, cycles or infrastructure leakage. |
| Security | Authorization bypass, CSRF, injection, traversal, SSRF, upload and session cases. |
| Performance | Query counts, memory, throughput and latency measured against budgets. |
| Compatibility | Supported PHP versions and MySQL/PostgreSQL/SQLite matrix remain green. |

## Recommended toolchain

- PHPUnit for unit/integration tests.
- PHPStan at a strict, progressively enforced level.
- PHP-CS-Fixer or equivalent formatter with committed rules.
- Composer audit and dependency update automation.
- PHPBench or repeatable benchmark harness for performance-sensitive components.
- Static architecture tests for package boundaries and forbidden dependencies.

```
Install → format check → static analysis → unit tests → integration matrix
→ security/dependency audit → architecture tests → benchmark smoke tests → package build
```

# 16. Performance and scalability

- Use OPcache in production; avoid expensive reflection or route compilation on every request.
- Compile/cache routes and configuration; invalidate safely on deployment.
- Choose indexes from observed query plans and use EXPLAIN workflows.

- Avoid unbounded eager loading, large in-memory collections and N+1 serialization.
- Use cursor/chunk processing for large datasets and stream large files.
- Move slow tasks to queues and keep request work bounded.
- Use shared external cache/session/queue infrastructure for multi-instance deployments.
- Publish reproducible benchmarks with hardware, PHP, database and configuration details.

# 17. Implementation roadmap and release gates

| Phase | Deliverables | Exit gate |
|---|---|---|
| 0 Foundation | Monorepo, Composer, standards, contracts and CI skeleton. | Packages autoload; architecture tests pass. |
| 1 Kernel | Config, container, providers, lifecycle and exceptions. | Boot and resolve services deterministically. |
| 2 CLI | Command bus, help, generators and doctor. | Stable command exit codes and tests. |
| 3 HTTP | Request/response, router, middleware and kernel. | HTTP integration tests pass. |
| 4 Database | PDO adapters, dialects, query builder, transactions and migrations. | Three-engine integration matrix passes. |
| 5 ORM | Models, casts, relationships and eager loading. | Relationship/query-count tests pass. |
| 6 Guardrails | Service/repository skeleton and N+1 diagnostics. | Architecture and N+1 fixtures pass. |
| 7 Security | Auth, policies, CSRF, encryption, rate limiting and audit. | Threat-focused tests and review pass. |
| 8 Operations | Cache, queue, events, scheduler and monitoring. | Failure/retry/worker tests pass. |
| 9 Integrations | Mail, notifications and storage. | Adapter contract tests pass. |
| 10 AI | Provider contracts, streaming, embeddings, RAG and tools. | Fake-provider and permission tests pass. |
| 11 Release | Docs, migration guides, examples and benchmarks. | Release checklist complete. |

## Suggested release milestones

- 0.1 kernel, container, config and CLI foundation; 0.2 HTTP routing/middleware; 0.3 database/query builder/migrations.
- 0.4 ORM and N+1 diagnostics; 0.5 security/authentication; 0.6 cache/queue/events/scheduler.
- 0.7 mail/notifications/storage/monitoring; 0.8 AI contracts and workflows; 0.9 beta hardening; 1.0 stable APIs and support policy.

# 18. Production-readiness checklist

- All supported PHP versions and all three database engines pass CI.
- No unresolved critical/high security defects; dependency audit is clean or documented with mitigations.
- Authentication, authorization, CSRF, session and token lifecycle tests are complete.
- Secrets are externalized; debug is disabled; errors do not leak internals.
- Migrations are tested on production-like versions; backup/restore has been rehearsed.
- N+1 detection, slow-query logs and query budgets work and redact sensitive data.
- Queue retries, timeouts, idempotency and failed-job recovery are tested and documented.
- AI tool access is permissioned, audited, budgeted and disabled unless configured.
- Health checks, logs, metrics and graceful shutdown are available.
- Public APIs, deprecations, upgrade path and security reporting process are documented.
- Performance claims are backed by repeatable benchmarks and realistic load tests.

# 19. Architecture decision records

Record major design decisions in docs/adr/ using a consistent template: context, decision, alternatives, consequences, security impact and migration plan. Initial ADRs should cover PDO as the first database foundation; monorepo versus split packages; PHP support policy; ORM lazy-loading policy; transaction boundaries; cache/queue priorities; AI provider contract; and semantic versioning.

## Recommended first engineering tasks

- Create the monorepo and package-level Composer manifests.
- Write package dependency rules and architecture tests before feature work.
- Define container, provider lifecycle, configuration and exception contracts.
- Build a minimal executable skeleton and one end-to-end HTTP route.
- Implement one database driver end-to-end, then generalize through dialect/contract tests.
- Add N+1 fixtures early so query behavior remains measurable.
- Create a threat model and security test plan before auth and upload features.
- Keep AI optional until core application, database and security foundations are stable.

---

**Closing note:** This document defines the target design for CnA Framework. Implement incrementally, preserve package boundaries, and make production claims traceable to tests, benchmarks or security-review evidence.